

Mesos

Final Software Engineering Project

Diana Francesconi, Luca Grecchi, Vadym Kitsul, Andrea Markvukaj

Java Maven application implementing MVC, JavaFX/TUI, Socket/RMI networking, persistence and a database-backed leaderboard.

Implemented Features

- Complete implementation of the Mesos game rules: players, tribes, cards, board, turns, phases and final scoring.
- Two client interfaces: **TUI** for terminal interaction and **JavaFX GUI** for graphical play.
- Client-server architecture with support for both **Socket** and **RMI**.
- Advanced feature: global match leaderboard stored in a PostgreSQL database.
- Advanced feature: game persistence and recovery after server crash.
- Final deliverables include client/server JARs, database dump, UML class diagram and sequence diagrams.

Architecture and Design Patterns

- **MVC architecture**

- model: domain rules, cards, board, players, phases and scoring.
- controller: server-side coordination and phase delegation.
- view: TUI and JavaFX GUI.

- **State pattern:** used both in the Controller and in the Model game flow. Controller behavior is split into LobbyPhase, InGamePhase and RecoveryPhase; the Model also delegates rules to phase objects such as totem placement, offer resolution, event resolution, extra card, end round and end game.
- **Observer-like flow:** the Model notifies the Controller through GameListener.
- **Repository pattern:** leaderboard logic depends on MatchResultRepository, implemented by SqlMatchResultRepository.
- **DTO-based communication:** snapshots and updates are transferred without exposing internal Model objects.

Communication and Advanced Features

- Shared abstractions: `VirtualServer` for client-to-server actions and `VirtualView` for server-to-client notifications.
- **Socket**: TCP connection on port 12345, serialized `ClientMessage` and `ServerMessage` objects, one handler per client.
- **RMI**: registry on port 1099, server bound as `MesosServer`, direct remote calls and client callbacks through `ClientRMI`.
- **Leaderboard**: stores nickname, final score, player count and match date; rankings are filtered by player count and keep each player's best result.
- **Persistence**: game state is serialized with `Gson`; polymorphic objects are restored through `InterfaceAdapter`.
- **Recovery**: after a crash, players reconnect using the original nickname and color within a 20-second timeout.

Testing and Coverage

- Test suite: **50 test classes** and **205 active JUnit tests** out of 206 methods annotated with @Test.
- Covered areas: Model, board, cards, game phases, controller phases, configuration, leaderboard, persistence utilities and selected network recovery behavior.
- Regression tests cover critical scenarios: optional building selection, optional extra-card selection, unavailable extra-card handling, round-end behavior, recovery timeout management and leaderboard consistency.
- Package-level coverage highlights:
 - **Model**: 95% class, 91% method, 91% line, 81% branch.
 - **Config**: 50% class, 100% method, 100% line, 83% branch.
 - **Controller**: 80% class, 55% method, 61% line, 39% branch.
 - **Leaderboard**: 100% class, 60% method, 45% line, 75% branch.
- Lower values mainly come from network-dependent flows, database-specific code, UI code and integration paths that are harder to exercise with isolated unit tests.